

HelixMemory

HelixMemory

Un cerveau mémoriel unique pour les agents AI — quatre moteurs de pointe, fusionnés.

Résumé

HelixMemory est un Go SDK qui unifie quatre systèmes mémoriels de premier plan (Mem0, Cognee, Letta, Graphiti) en un seul moteur cognitif de mémoire, capable de les interroger en parallèle et de fusionner les résultats. Il offre aux applications AI une couche mémorielle unique, durable, dédoublée et reclassée, au lieu de quatre couches disjointes.

Description courte

HelixMemory est un Go SDK qui fusionne Mem0, Cognee, Letta et Graphiti en un moteur cognitif de mémoire unifié pour les applications AI. Il achemine intelligemment les écritures, interroge tous les backends en parallèle et fusionne les résultats via un pipeline en trois étapes : collecte, déduplication et reclassement.

Description longue

HelixMemory est un moteur cognitif de mémoire unifié pour les applications AI, fourni sous la forme d'un Go SDK (module `digital.vasic.helixmemory`, Go 1.25+). Son postulat fondateur est qu'aucun projet de mémoire ne sera jamais le meilleur en tout — plutôt que de réimplémenter la mémoire à partir de zéro et d'hériter des angles morts d'un seul projet, il orchestre quatre systèmes de pointe et permet à chacun d'exploiter ses forces : Mem0 pour l'extraction dynamique de faits et la gestion des préférences, Cognee pour les graphes de connaissances sémantiques construits via des pipelines ECL, Letta pour un runtime d'agent à mémoire modifiable avec des blocs de mémoire

éditables et un calcul en arrière-plan, et Graphiti pour un graphe de connaissances bi-temporel capable de raisonner sur l'évolution des faits dans le temps.

Un moteur de fusion est ce qui transforme ces quatre systèmes indépendants en un seul cerveau. Lors de l'écriture, chaque nouvelle mémoire est classée selon son contenu et acheminée vers le backend le plus adapté pour la stocker. Lors de la lecture, une requête est diffusée en parallèle à tous les backends, et les résultats bruts sont traités par un pipeline de fusion en trois étapes — collecte, déduplication, puis reclassement inter-sources — de sorte que l'appelant ne voit jamais quatre ensembles de résultats bruyants et redondants, mais une seule réponse propre et classée. Des disjoncteurs enveloppent chaque backend pour une dégradation gracieuse : lorsqu'un moteur tombe en panne, son disjoncteur se déclenche et les backends restants continuent de fonctionner, évitant ainsi d'entraîner toute la couche mémorielle dans sa chute. Comme le moteur implémente une interface MemoryStore prête à l'emploi, il s'intègre directement en remplacement d'un fournisseur de mémoire standard — sans nécessiter de réarchitecture côté appelant — et les métriques Prometheus exposent les rouages internes du routage et de la fusion pour une observabilité totale.

HelixMemory a été conçu comme couche mémorielle pour HelixAgent, l'ensemble AI Helix élargi, et il applique à la mémoire la discipline de test anti-bullshit de la famille : un exécuteur de défis intégré active les véritables chemins de code en production — routage, fusion, traducteur, disjoncteur — tandis qu'un wrapper de mutation apparié inverse délibérément les invariants pour prouver que les tests échouent effectivement lorsque la logique est défaillante, garantissant ainsi qu'une suite de tests au vert signifie quelque chose.

Pourquoi nous l'avons conçu

Contenu

Les agents AI nécessitent une mémoire durable et de haute qualité, mais l'écosystème est fragmenté — chaque projet de mémoire (Mem0, Cognee, Letta, Graphiti) excelle dans un domaine tout en présentant des lacunes dans d'autres. HelixMemory a été conçu pour offrir à HelixAgent une surface mémoire unifiée qui combine leurs forces sans imposer un verrouillage sur l'un d'eux.

Pourquoi c'est révolutionnaire

Il met fin au choix forcé. Quatre systèmes de mémoire qui se disputent habituellement le même créneau deviennent des backends complémentaires derrière une seule interface — ainsi, une application bénéficie simultanément d'une extraction dynamique de faits, de graphes de connaissances sémantiques, d'une mémoire d'agent à état persistant et d'un raisonnement bi-temporel, avec une déduplication et un réordonnancement inter-sources gérés automatiquement. Ce qui était auparavant irréalisable — considérer « quel moteur de mémoire adopter ? » comme un faux dilemme — devient possible : HelixMemory vous permet de cumuler leurs atouts en une seule MemoryStore prête à l'emploi, sans hériter des angles morts d'un moteur ni s'engager dans un verrouillage.

Ce qui est innovant

- Une **fusion multi-backend** (collecte → déduplication → réordonnancement inter-sources) qui renvoie un ensemble de résultats classés, plutôt que d'imposer au client un seul magasin.
- Un **routage intelligent des écritures** qui classe chaque mémoire en fonction de son contenu et l'envoie vers le moteur le mieux adapté pour la stocker, garantissant que les bonnes données atterrissent au bon endroit.
- Une **dégradation gracieuse** via des disjoncteurs par backend — un moteur défaillant est isolé, sans être fatal, et les autres continuent de répondre.
- Une **consolidation des calculs en période d'inactivité** (via Letta) qui retravaille la mémoire pendant les phases creuses plutôt qu'uniquement au moment des requêtes.
- Une **vérification anti-bluffer** : un exécuteur de défis sur du code de production réel, couplé à un wrapper de mutation qui doit échouer lorsqu'un invariant est inversé — prouvant que la porte de test est un véritable contrôle, et non une tautologie.

Principaux défis techniques et solutions apportées

- **Réconcilier quatre backends hétérogènes en un ensemble de résultats cohérent** — chaque moteur renvoie la mémoire dans son propre format, et une fusion naïve génère des doublons et des classements incomparables. Résolu grâce à un moteur de fusion typé qui collecte les sources, déduplique les chevauchements et réordonne l'ensemble sur une base commune, avec un invariant de comptage fusionné vérifié dans

les tests pour éviter toute perte ou double comptage silencieux.

- **Rester opérationnel lorsque qu'un backend tombe en panne** — un moteur de mémoire inaccessible ne doit pas bloquer toute la couche. Résolu avec des disjoncteurs par backend qui suivent un automate d'états fermé → ouvert (après un seuil d'échec) → semi-ouvert (après un délai), isolant le backend défaillant et continuant à servir depuis les autres jusqu'à son rétablissement.
- **Prouver que la logique mémoire fonctionne réellement, et pas seulement qu'elle compile** — une suite de tests au vert est inutile si les tests ne peuvent pas échouer. Résolu avec un exécuter de défis intégré qui pilote le code de production réel (routage, fusion, traducteur, disjoncteur) et un wrapper de mutation couplé qui inverse les invariants et exige que les tests passent au rouge, garantissant ainsi que la porte de contrôle n'est pas une simple tautologie.

Contenu

Pile technologique

- **Go (1.25+)** — le SDK et son environnement d'exécution unifiés ; choisi car la lecture parallèle en éventail sur quatre backends pose un problème de concurrence, et les goroutines de Go en rendent le coût négligeable, tandis que ses types d'interface offrent à l'ensemble du système une seule couture propre (MemoryStore) sur laquelle les appelants peuvent s'appuyer.
- **Mem0** — le backend d'extraction dynamique de faits et de gestion des préférences ; utilisé pour la partie « quelles sont les préférences réelles de cet utilisateur / quels faits ont émergé » de la mémoire.
- **Cognee** — le backend de graphe de connaissances sémantique construit sur des pipelines ECL ; utilisé pour stocker des connaissances structurées et liées, plutôt que des faits isolés.
- **Letta** — le backend d'exécution d'agents à état persistant, doté de blocs de mémoire modifiables et de calcul en arrière-plan pendant les périodes d'inactivité ; utilisé lorsque la mémoire doit persister en tant qu'état actif de l'agent et être consolidée durant les phases d'inactivité.
- **Graphiti** — le backend de graphe de connaissances bi-temporel ; utilisé pour raisonner sur l'évolution des faits et des relations dans le temps, et non seulement sur leur valeur actuelle.
- **PostgreSQL + Neo4j + Redis** — les véritables datastores sur lesquels s'exécutent les backends, déployés pour des tests d'intégration réels via `make infra-start`, afin que la suite teste une infrastructure en conditions réelles plutôt que des simulacres.

- **Prometheus** — métriques et observabilité intégrées via le pipeline de fusion, permettant de mesurer en production le comportement de routage et de fusion, plutôt que de fonctionner en boîte noire.
- **Interface de traduction i18n** — une surface de chaînes de caractères organisée par espaces de noms (`helixmemory_`), maintenue en place pour qu'une future couche utilisateur puisse être localisée sans avoir à modifier le cœur du système.

Statut et notes de transparence

- **Statut : bêta.** SDK fonctionnel ; conçu comme couche mémoire pour HelixAgent.
- **Licence : à déterminer.** Aucune LICENCE n'a été détectée via le GitHub API — NON VÉRIFIÉE / non déclarée.
- Le nom d'affichage « HelixMemory » correspond au dépôt `memory`. Les chiffres de précision mentionnés dans le README proviennent des affirmations des fournisseurs en amont, et non de mesures effectuées par HelixMemory, et sont donc omis ici.

Niveau de priorité : Helix-primaire.